

اصل ISP می‌گوید:

"هیچ کلاسی نباید مجبور شود متدهایی را پیاده‌سازی کند که به آنها نیازی ندارد."

به عبارت دیگر، بهتر است اینترفیس‌ها کوچک و تخصصی باشند، نه بزرگ و عمومی، تا کلاس‌ها فقط متدهایی را پیاده‌سازی کنند که واقعا استفاده می‌کنند.

مثال نقض ISP

فرض کنید یک اینترفیس داریم:

```
public interface IMachine
{
    void Print();
    void Scan();
    void Fax();
}
```

و حالا یک کلاس OldPrinter داریم که فقط چاپ می‌کند:

```
public class OldPrinter : IMachine
{
    public void Print()
    {
        Console.WriteLine("Printing...");
    }

    public void Scan()
    {
        // اسکن ندارد OldPrinter اینجا چکار کنیم؟
        throw new NotImplementedException();
    }

    public void Fax()
    {
        // فکس ندارد OldPrinter این هم چکار کنیم؟
        throw new NotImplementedException();
    }
}
```

مشکل اینجاست که OldPrinter مجبور شد متدهای ()Scan و ()Fax را پیاده‌سازی کند، در حالی که استفاده‌ای از آنها ندارد. این نقض ISP است.

اصلاح مثال

راهکار این است که اینترفیس را به چند اینترفیس کوچک‌تر تقسیم کنیم:

```
public interface IPrinter
{
    void Print();
}

public interface IScanner
{
    void Scan();
}

public interface IFax
{
    void Fax();
}
```

حالا کلاس OldPrinter فقط اینترفیس IPrinter را پیاده‌سازی می‌کند:

```
public class OldPrinter : IPrinter
{
    public void Print()
    {
        Console.WriteLine("Printing...");
    }
}
```

اگر دستگاهی هم چاپ و اسکن دارد، می‌تواند چند اینترفیس را با هم پیاده‌سازی کند:

```
public class AllInOnePrinter : IPrinter, IScanner, IFax
{
    public void Print() { /* ... */ }
    public void Scan() { /* ... */ }
    public void Fax() { /* ... */ }
}
```

☑ نتیجه: با تقسیم اینترفیس‌های بزرگ به اینترفیس‌های کوچک، کلاس‌ها مجبور به پیاده‌سازی متدهای اضافی نمی‌شوند و هر کلاس فقط متدهایی را دارد که واقعا نیاز دارد.